

TrazaRed HUV — Documentación técnica: mapeo BD → FHIR R4 y justificación de roles

Versión 1.0 · 6 de septiembre de 2026 · Corresponde al requisito de *documentación técnica* del proyecto (10 % de la rúbrica del Corte 1).

Este documento ajusta las **tablas 6 a 9 del plan de acción** a lo que quedó **implementado** en `python/fhir_sync.py` y `python/main.py`, y deja registradas las desviaciones y su justificación (§7).

1. Alcance y arquitectura

Componente	Tecnología	Contenido
BD relacional	PostgreSQL (Neon)	6 tablas: pacientes, usuarios, remisiones, observaciones, remisiones_historial, auditoria (ver db/schema.sql)
BD documental	MongoDB (Atlas)	1 colección: trazared_huv.gestiones_contacto
API	FastAPI + JWT	Autenticación, roles, CRUD, soft delete/edit/restauración (python/main.py)
Servidor FHIR	HAPI FHIR R4 (Docker)	Recursos clínicos interoperables, publicado en el puerto 8081 del host
Integración	python/fhir_sync.py	Mapeo fila → recurso y POST/GET contra HAPI

2. Mapeo implementado campo a campo

2.1 pacientes → Patient (tabla 6 del plan, ajustada)

Campo BD	Elemento FHIR R4	Tipo / terminología	Notas
documento	identifier[0] con system = urn:trazared:documento	Identifier	Identificador nacional del paciente en un sistema propio trazable
nombre	name[0].text	HumanName	Nombre completo tal como está en la BD
genero	gender	AdministrativeGender (male / female / unknown)	F→female, M→male, cualquier otro→unknown
eps	— (sin mapeo)	—	El plan contemplaba el recurso Coverage; ver §3
id	— (no viaja en el POST)	—	HAPI asigna el id definitivo del recurso; el id de la BD queda trazable por identifier

Diferencias con la tabla 6 del plan: el modelo simplificado de pacientes no captura tipo_documento, fecha_nacimiento, direccion ni telefono, por lo que los elementos birthDate, address y telecom del Patient

no se poblaron (mejora futura si el formulario de ingreso los agrega). El soft delete de pacientes no existe en este corte, así que active no se utiliza.

2.2 remisiones → Encounter (*tabla 7 del plan, ajustada*)

Campo BD	Elemento FHIR R4	Tipo / terminología	Notas
estado	status	EncounterStatus	pendiente→planned, en_gestion→arrived, aceptada→in-progress, rechazada→cancelled, completada→finished
— (fijo)	class	Coding v3-ActCode TRNS (<i>transfer</i>)	Toda remisión es un traslado entre instituciones; la BD no tiene columna de clase de servicio
paciente_id	subject = Patient/{id_fhir}	Reference	Usa el id asignado por HAPI al sincronizar el paciente, no el id de la BD
motivo	reasonCode[0].text	CodeableConcept	Motivo en texto libre (la BD no guarda código CIE-10)
fecha_solicitud	period.start	date	Inicio de la ventana de la remisión
institucion_origen / institucion_destino	— (sin mapeo)	—	Requerirían Organization (§3)
convenio_vigente, creado_por, activo	— (sin mapeo)	—	Metadatos operativos propios de la BD; ver §7

Diferencias con la tabla 7 del plan: el mapa de estados implementado cubre los cinco estados del CHECK de la BD (el plan no contemplaba en_gestion→arrived ni rechazada→cancelled). motivo_cie10, clase_servicio, medico_id y destino_id del plan no existen como columnas en el esquema implementado.

2.3 observaciones → Observation (*tabla 8 del plan, ajustada*)

Campo BD	Elemento FHIR R4	Tipo / terminología	Notas
— (fijo)	status = final	ObservationStatus	Valores registrados como definitivos
codigo_loinc	code.coding[0] con system = http://loinc.org	Coding LOINC	Ej.: 8867-4 frecuencia cardíaca, 8480-6/8462-4 presión arterial, 8310-5 temperatura, 59408-5 saturación
tipo	code.text	CodeableConcept	Descripción legible; queda sola si no hay código LOINC
valor	valueQuantity.value	Quantity	Valor numérico del signo vital
unidad	valueQuantity.unit	Quantity	Unidad de medida (UCUM cuando aplica)
(por la remisión)	subject = Patient/{id_fhir}	Reference	Paciente al que pertenece

Campo BD	Elemento FHIR R4	Tipo / terminología	Notas
remision_id	encounter = Encounter/{id_fhir}	Reference	Remisión en cuyo contexto se registró
fecha_observacion	— (sin mapeo)	—	Mejora futura: effectiveDateTime

Diferencias con la tabla 8 del plan: el esquema implementado solo tiene valor numérico (no booleano/texto), no tiene estado propio ni medico_id, por lo que no se poblaban valueBoolean, valueString, performer ni estados amended / entered-in-error.

3. Recursos adicionales del plan (tabla 9) no implementados

Recurso plan	Origen previsto	Decisión tomada	Justificación
Organization	instituciones / EPS	No implementado	Alcance priorizado en 3 recursos; las instituciones viven como texto en remisiones
Coverage	pacientes.eps_id	No implementado	No hay tabla de EPS; la EPS es un TEXT en pacientes
Practitioner	usuarios médicos	No implementado	creado_por queda en la BD y en auditoria; exponerlo en FHIR no aporta al caso de uso del corte
Task	gestiones de contacto	Sustituido por MongoDB	La bitácora de contactos se modela como documento anidado (§4): encaja mejor con el requisito del curso de usar ambos motores (persistencia polígota) y evita la rigidez de Task.input/output

4. gestiones_contacto en MongoDB (Atlas)

Colección trazared_huv.gestiones_contacto. Un documento por remisión, con la bitácora de intentos anidada — responde directamente la pregunta del caso HUV (*quién contactó, a quién, cuándo, con qué respuesta*):

```
{
  "remision_id": 1,
  "contactos": [
    {
      "fecha": "2026-09-06T15:30:00",
      "medio": "telefono",
      "institucion_contactada": "Hospital Departamental",
      "contactado_por": "Medico Prueba",
      "respuesta": "Sin cupo en UCI"
    }
  ],
  "actualizado_en": "2026-09-06T15:31:02"
}
```

5. Matriz de endpoints × roles (implementada en python/main.py)

Todas las rutas excepto POST /login exigen JWT (Authorization: Bearer ...). Las operaciones de soft edit / soft delete / restauración dejan registro en auditoría.

Endpoint	admin	medico	eps	paciente
---	:-:	:-:	:-:	---
POST /login	público — devuelve el JWT			
POST /usuarios	✓	✗	✗	✗
GET/POST /pacientes	✓	✓	✗	✗
GET /remisiones	✓ todas	✓ todas	✓ solo remisiones cuyos pacientes tengan su EPS (p.eps = eps_nombre)	✓ solo las suyas, con 4 campos (id, institucion_destino, fecha_solicitud, estado)
GET /remisiones/{id}	✓	✓	✓ solo si es de su EPS (403 si no)	✓ solo si es suya (404 si no), campos reducidos
POST /remisiones	✓	✓ (queda como creado_por)	✗	✗
PUT /remisiones/{id} (soft edit: guarda el estado previo en remisiones_historial)	✓	✓	✗	✗
DELETE /remisiones/{id} (soft delete)	✓	✓ solo si la creó él (403 si no)	✗	✗
POST /remisiones/{id}/restaurar	✓ único rol	✗	✗	✗
GET/POST /observaciones	✓	✓	✗	✗
GET/GET {id}/POST/PUT /gestiones-contacto	✓	✓	✗	✗
DELETE /gestiones-contacto/{id}	✓ único rol	✗	✗	✗

6. Justificación de los cuatro roles

El plan original contemplaba tres roles; se agregó paciente (desviación documentada en §7). La definición de cada uno sigue el principio de mínimo privilegio:

1. admin — **operación del sistema**. Gestiona usuarios (POST /usuarios) y es el único que puede **restaurar** remisiones eliminadas y **borrar definitivamente** gestiones de contacto. La restauración y el borrado duro quedan concentrados en un rol de confianza (separación de deberes): ningún rol clínico puede deshacer auditoría. 2. medico — **rol clínico**. Crea pacientes, remisiones y observaciones; edita con trazabilidad (soft edit) y elimina lógicamente **solo las remisiones que él mismo creó** (creado_por), lo que se verifica con 403 en caso contrario. 3. eps — **contraparte pagadora**. Solo lectura, y únicamente de las remisiones de sus afiliados. No ve usuarios, no ve observaciones clínicas (los signos vitales quedan restringidos a admin/médico) y no puede crear ni alterar nada: la EPS responde, no registra. 4. paciente — **titular del dato**. Se añadió frente al plan para cerrar el ciclo de la Sentencia T-573-23 (transparencia hacia el usuario): el paciente

consulta **sus** remisiones con una proyección reducida (destino, fecha, estado) sin exponer campos operativos internos como convenio_vigente, creado_por o motivo.

7. Desviaciones respecto al modelo de la Semana 3 (y por qué se mantienen)

#	Plan (Semana 3)	Implementado	Justificación
1	3 roles (admin, medico, eps)	4 roles (+ paciente)	Evidencia el filtrado por dueño del dato y responde al caso de la T-573-23 (§6)
2	10 tablas	6 tablas + MongoDB	Las gestiones de contacto son naturales de BD documental; el modelo relacional se limitó a lo que el CRUD realmente usa
3	7 recursos FHIR	3 recursos (Patient, Encounter, Observation)	Cubren el flujo clínico mínimo verificable; el resto quedó documentado en §3
4	Carpetas sql/ api/ etl/ hapi/ docs/	db/ docker/ python/ notebooks/	Estructura real del repo al momento de la integración
5	Tabla mapeo_fhir de idempotencia	Sin tabla	La trazabilidad se logra con identifier propio + verificación GET; re-ejecutar la sincronización crea recursos nuevos en HAPI (comportamiento conocido y aceptado para el corte)

8. Cómo reproducir la sincronización y verificarla

```
# 1. Levantar HAPI FHIR (queda en http://localhost:8081/fhir)
cd docker && docker compose up -d

# 2. Desde python/ (requiere python/pass.env – copia del pass.env de la raíz)
cd ../python && python fhir_sync.py
# ...o ejecutar las celdas de la sección 8 del notebook.

# 3. Verificar con GET (navegador o curl) – sección 8c del notebook
curl http://localhost:8081/fhir/Patient/{id}
curl http://localhost:8081/fhir/Encounter/{id}
curl http://localhost:8081/fhir/Observation/{id}
```